

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	5
O QUE VOCÊ VIU NESTA AULA?	14
REFERÊNCIAS.....	15

EMENDAS

O QUE VEM POR AÍ?

Quando pensamos em avaliação de modelos, será que é confiável olhar apenas para a acurácia do modelo? Te garanto que existem muito mais técnicas para validar se ele é confiável para subirmos em produção.

Assim, nesta aula você vai aprender duas técnicas importantes para validar modelos de aprendizagem supervisionada de forma robusta: a validação cruzada e o método de bootstrapping!

EMEND

HANDS ON

Nesta aula você vai aprender na prática como utilizar essas ferramentas poderosas e robustas de avaliação de modelos no Python. Vamos lá?

EMSE

SAIBA MAIS

Conhecendo a validação cruzada

Para falar sobre validação cruzada, devemos lembrar como é realizada a técnica de amostragem hold out, em que basicamente se realiza a separação da amostra de treino e teste da base de dados antes de executar um modelo de aprendizado supervisionado.

A separação da base consiste entre 70% ~80% para treinar o algoritmo e o resto da base entre 20% ~30% para teste. Basicamente, com essa amostragem maior de dados, o algoritmo deve aprender os padrões nos dados para posteriormente testarmos se essa aprendizagem em uma base de dados nunca vista anteriormente (a base de teste faz esse papel de “novos dados”) tem efeito positivo.

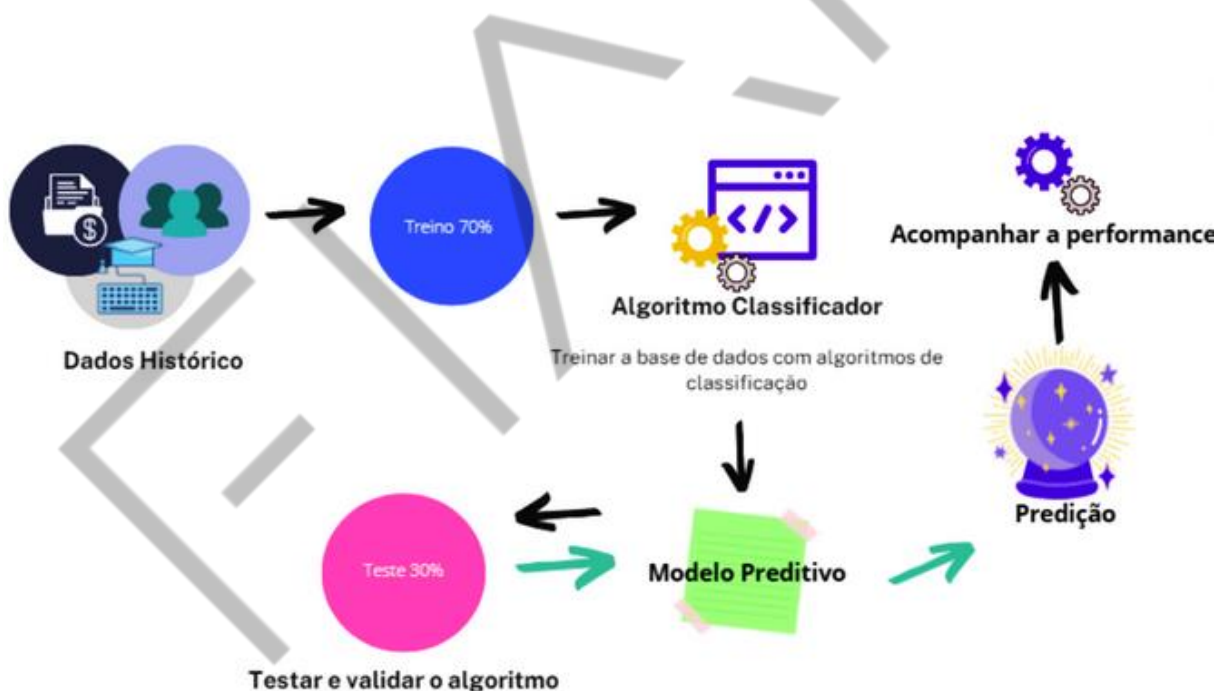


Figura 1 - Separação de dados de forma tradicional para treinamento de algoritmos (técnica de hold out)

Fonte: Elaborado pela autora (2024)

A validação cruzada consiste em uma técnica mais robusta do que essa simples estratificação dos dados em treino e teste. Vamos entender como funciona a validação cruzada e qual é a principal diferença!

A validação cruzada, também conhecida como k-fold cross-validation, divide de forma aleatória os dados de um conjunto de treinamento em subconjuntos distintos (os chamados folds). Ela então treina e avalia o modelo em cada subconjunto de dados, obtendo resultados diferentes de acordo com a iteração de K escolhida.

k=1	k=2	k=3	k=4
Test	Train	Train	Train
Train	Test	Train	Train
Train	Train	Test	Train
Train	Train	Train	Test

Figura 2 - Técnica de validação cruzada (técnica de k-folds cross validation)
Fonte: Elaborado pela autora (2024)

O grande **objetivo dessa técnica é avaliar a performance e a generalização** de um modelo de aprendizado de máquina. Mas por que ela é considerada robusta? Ao separar os dados em vários mini conjuntos de dados (os chamados k-folds), o treinamento e a validação do modelo são realizados de forma iterativa.

Como os dados são divididos em subconjuntos, é como se você estivesse treinando o seu modelo em várias bases de dados diferentes nesses conjuntinhos de k-folds. Imagine que você separou seu conjunto de dados em cinco folds, sendo que quatro deles serão utilizados para treinar o modelo e um para validar.

Você pode selecionar uma métrica de validação, como o f1-score, para validar os conjuntos de k-folds de forma agregada a fim de selecionar uma medida mediana, por exemplo, da performance do conjunto geral de todos os k-folds. Se analisar todas as amostras e o resultado médio for bom, significa que seu modelo está sabendo **generalizar** bem os dados.

A validação cruzada visa **evitar o overfitting** (decoração dos dados de treinamento) e construir modelos mais confiáveis e robustos. Se a performance do modelo foi significativamente melhor no conjunto de treinamento do que no conjunto de validação, isso pode indicar que o modelo está sofrendo de overfitting.

A estimativa de performance gerada pela avaliação do modelo de forma agregada pode se tornar muito robusta, uma vez que leva em consideração a **variabilidade entre os diferentes k-folds do conjunto de dados**. Observe uma aplicação da validação cruzada utilizando o Python:

```
import numpy as np
import pandas as pd

from sklearn.model_selection import KFold
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error

# Gera dados aleatórios
np.random.seed(42)
X = np.random.rand(100, 2)
y = np.round(X[:, 0] + X[:, 1] + np.random.randn(100))

# Cria DataFrame
df = pd.DataFrame({'characteristical': X[:, 0], 'caracteristica2': X[:, 1], 'classe': y})

# Configura validação cruzada com 5 folds
kfold = KFold(n_splits=5, shuffle=True, random_state=42)

# Inicializa modelo
model = LinearRegression()

# Armazena scores e desvios padrão
scores = []
std_devs = []

# Itera sobre os folds
for train_index, test_index in kfold.split(X):
    X_train, X_test = X[train_index], X[test_index]
```

```
y_train, y_test = y[train_index], y[test_index]

# Treina o modelo no conjunto de treinamento
model.fit(X_train, y_train)

# Faz previsões no conjunto de teste
y_pred = model.predict(X_test)

# Calcula o erro médio absoluto
mae = mean_absolute_error(y_test, y_pred)

# Armazena scores e desvios padrão
scores.append(mae)
std_devs.append(np.std(y_pred - y_test))

# Mostrando os resultados
print("Scores da validação cruzada:")
print(scores)

print("\nDesvios padrão:")
print(std_devs)

print("\nMAE médio:", np.mean(scores))
print("Desvio padrão do MAE:", np.std(scores))
```

Figura 3 - Aplicação da validação cruzada utilizando o Python
Fonte: Elaborado pela autora (2024)

Mostrando os resultados do modelo com validação cruzada:


```
Scores da validação cruzada:  
[0.7029154837311256, 0.8057328871752519, 0.8751863699639845, 0.9727943328042411, 0.9007801035080109]  
  
Desvios padrão:  
[0.8005064919733853, 1.0539908019578184, 0.9273080028361886, 1.2565773623847523, 1.0770272650128445]  
  
MAE médio: 0.8514818354365229  
Desvio padrão do MAE: 0.09151379495537142
```

Figura 4 - Resultado do modelo de regressão linear utilizando a validação cruzada
Fonte: Elaborado pela autora (2024)

Observe que, com o resultado da análise média das métricas de validação em todos os conjuntos gerados em k-folds, a capacidade de generalização do modelo se torna muito maior.

Como a ideia de modelos de aprendizado de máquina é aprender uma tarefa de forma automática, um dos grandes desafios do aprendizado de máquina é garantir que o modelo não “decore” os dados de treinamento e saiba identificar o padrão de dados novos, nunca visto antes pelo modelo. Essa técnica de validação cruzada é essencial para assegurar que ele tem grande chance de performar bem com novos dados.

Conhecendo a técnica de bootstrapping

Outra técnica avançada de avaliação de modelos de machine learning é a bootstrap. Essa técnica é uma poderosa ferramenta utilizada para avaliar a performance e a robustez de modelos de aprendizado de máquina. Imagine que ao selecionar um modelo de machine learning para treinar o padrão nos dados, o bootstrap utiliza todo o conjunto de dados original para serem criadas diversas amostras menores, chamadas de amostras bootstrap (múltiplas amostras).

Quando a amostragem é realizada com substituição, ou seja, os pontos podem ser selecionados mais de uma única vez no processo, este método é chamado de bagging (abreviação de bootstrap aggregating). Quando a amostragem é realizada sem substituição, é chamado de pasting. Cada amostragem da estratificação do bootstrap é realizada de forma aleatória.

No fim do treinamento, os resultados de performance de todos os modelos bootstrap são então agregados para obter uma estimativa mais robusta da performance do modelo original.

O bootstrap fornece uma estimativa da variabilidade da performance do modelo original, o que permite avaliar a robustez do modelo em relação a diferentes conjuntos de dados. Se a variabilidade entre as amostras bootstrap for alta, isso pode indicar que o modelo está sofrendo de overfitting, ou seja, está muito ajustado ao conjunto de dados de treinamento e não generaliza bem para novos dados.

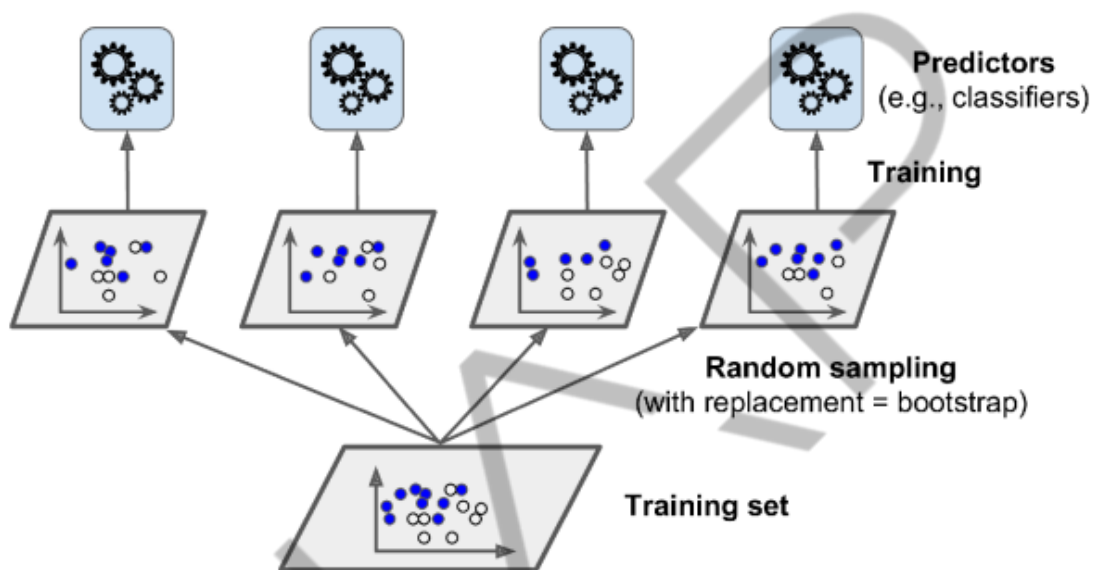


Figura 5 - Funcionamento do bootstrapping
Fonte: Géron (2019)

Uma vez que todos os previsores são treinados, o modelo pode realizar uma previsão para uma nova instância simplesmente agregando as previsões de todos os previsores. A função de agregação seleciona a previsão mais frequente, assim como um classificador hard voting para a situação de classificação e a média para regressão.

Veja uma aplicação da técnica de bootstrapping utilizando o sklearn:

```
import numpy as np

from sklearn.model_selection import train_test_split

from sklearn.ensemble import BaggingClassifier

from sklearn.tree import DecisionTreeClassifier

from sklearn.datasets import make_classification

# Gerando dados fake

X, y = make_classification(n_samples=100, n_features=4,
n_informative=2, n_redundant=0, random_state=0, shuffle=False)

# Separando os dados em treino e teste

X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2)

bag_clf = BaggingClassifier(
    DecisionTreeClassifier(), n_estimators=500,
bootstrap=True, n_jobs=-1, oob_score=True)

bag_clf.fit(X_train, y_train)

y_pred = bag_clf.predict(X_test)

from sklearn.metrics import accuracy_score

accuracy_score(y_test, y_pred)
```

Figura 6 - Aplicação da técnica de bootstrapping utilizando o sklearn
Fonte: Elaborado pela autora (2024)

Analisando os resultados, temos:

```
[46] from sklearn.metrics import accuracy_score  
accuracy_score(y_test, y_pred)  
  
0.9
```

Figura 7 - Resultado do modelo utilizando BaggingClassifier
Fonte: Elaborado pela autora (2024)

Avaliação Out-of-Bag

Com a técnica de bagging, algumas instâncias podem ser amostradas muitas vezes em qualquer previsor, enquanto outras podem simplesmente não serem amostradas. Por padrão, um BaggingClassifier do sklearn utiliza “m” instâncias de treinamento com substituição (bootstrap=True), sendo “m” o tamanho da base de dados.

Isso significa que somente 63% das instâncias de treinamento são amostradas na média para cada previsor. Os 37% restantes das instâncias que não são amostradas são chamados de instâncias out-of-bag (oob). Observe que elas são os mesmos 37% para todos os previsores. Como um previsor nunca vê as instâncias out-of-bag (oob) durante o treinamento, ela pode ser avaliada nessas instâncias sem a necessidade de um conjunto de validação separado ou de validação cruzada.

Observe no código da figura 7, que o parâmetro out-of-bag está configurado como oob_score=True. Sendo assim, você pode acessar a pontuação da avaliação resultante. Agora, na figura 8, o resultado está bem próximo da acurácia do conjunto de teste. Legal né?

```
bag_clf.oob_score_  
  
0.875
```

Figura 8 - Resultado da avaliação out-of-bag
Fonte: Elaborado pela autora (2024)

Que tal dar uma olhadinha na documentação oficial do Sklearn para conhecer mais os parâmetros do bootstrapping e da validação cruzada? É só clicar [aqui](#) e [aqui](#).

Ambas as técnicas são utilizadas para gerar uma **avaliação mais robusta dos modelos de machine learning e fornecer estimativas de erro de generalização**. No entanto, por mais que sejam técnicas que trabalham múltiplos conjuntos de dados aleatórios, elas possuem suas particularidades:

Característica	Bootstrap	Validação Cruzada
Objetivo	Estimar a distribuição de uma estatística, detectar outliers, agrupamento de dados	Avaliar o erro de generalização, selecionar hiperparâmetros, comparar modelos
Tipo	Técnica de reamostragem	Técnica de avaliação de modelo
Uso de dados	Gera novos conjuntos de dados a partir do original	Divide o conjunto de dados original em subconjuntos
Estimativa de incerteza	Sim	Sim
Seleção de variáveis	Sim	Não
Deteção de outliers	Sim	Não
Agrupamento de dados	Sim	Não
Precisão da estimativa do erro de generalização	Menor	Maior
Custo computacional	Menor	Maior

Figura 9 - Diferenças entre Bootstrap e validação cruzada
Fonte: Elaborado pela autora (2024)

O QUE VOCÊ VIU NESTA AULA?

Nessa aula você aprendeu quais são as principais técnicas de avaliação robusta de modelos de machine learning.

Assim, Bootstrap e Validação Cruzada são ferramentas complementares para avaliar o desempenho de modelos de aprendizado de máquina, fornecendo insights valiosos sobre a capacidade de generalização do modelo e auxiliando na tomada de decisões importantes para o desenvolvimento de modelos robustos e eficazes.

REFERÊNCIAS

GÉRON, A. **Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow**. 2. ed. [s.l.]: O'Reilly Media, Inc., 2019.

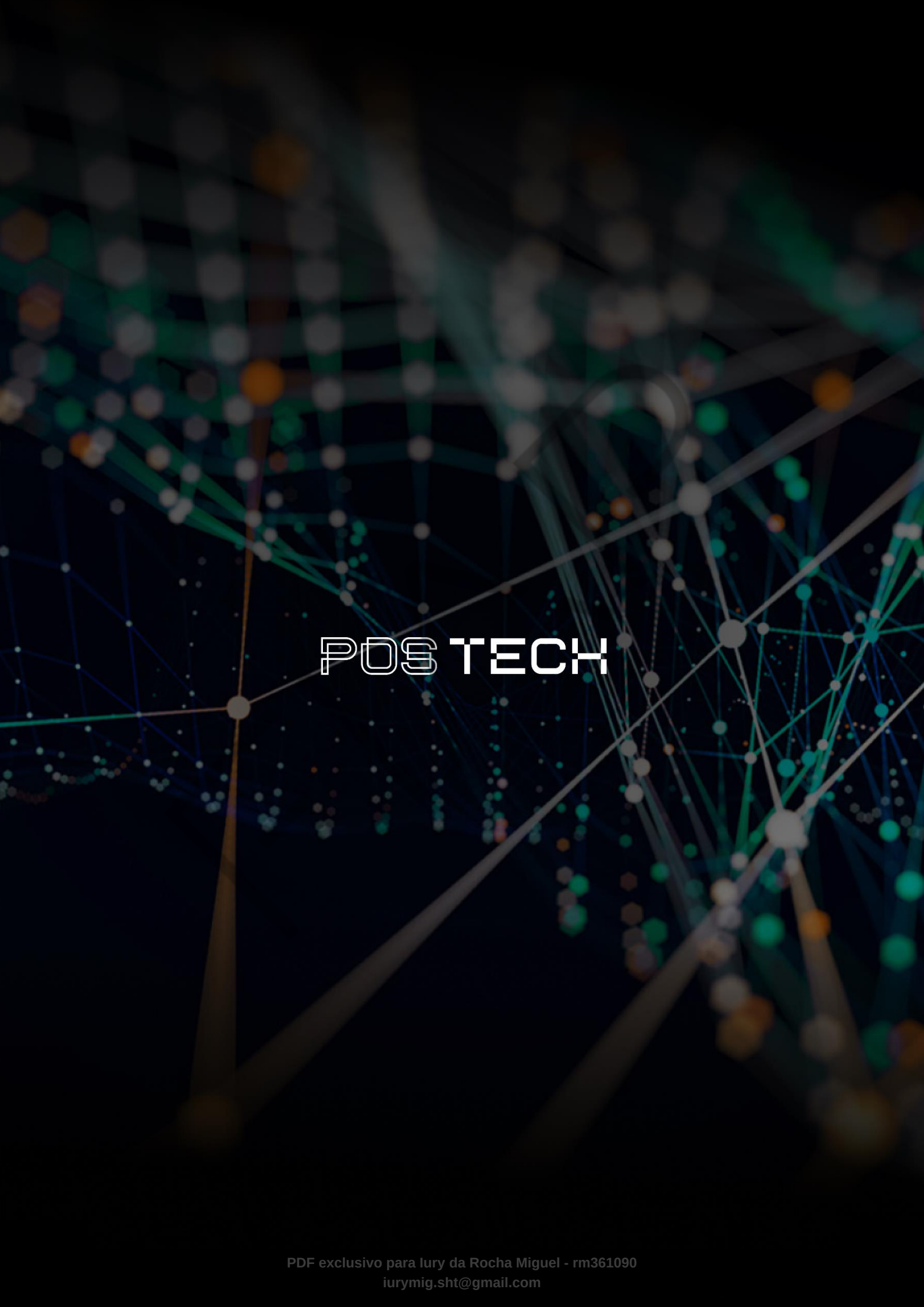
SCIKIT LEARN. **BaggingClassifier**. 2016. Disponível em: <<https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.BaggingClassifier.html>>. Acesso em: 18 jun. 2024.

SCIKIT LEARN. **KFold**. [s.d.]. Disponível em: <https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.KFold.html>. Acesso em: 18 jun. 2024.

PALAVRAS-CHAVE

Palavras-chave: Bootstrapping. Bagging. Validação cruzada. K-fold.

EMAP



POSTECH